

SQL — Fiche de révision

Concours ANP — Cadre Data Scientist

1 Requête de base : SELECT, WHERE

```
-- Sélectionner des colonnes précises
SELECT nom, tirant_eau
FROM navires;

-- Sélectionner toutes les colonnes
SELECT * FROM navires;

-- Filtrer les lignes avec WHERE
SELECT nom, tirant_eau
FROM navires
WHERE tirant_eau > 14;

-- Combiner des conditions
SELECT * FROM navires
WHERE port = 'Casablanca' AND tirant_eau > 12;

-- Filtrer sur un ensemble de valeurs ou un motif
SELECT * FROM navires WHERE port IN ('Tanger', 'Agadir');
SELECT * FROM navires WHERE nom LIKE 'MSC%';
SELECT * FROM navires WHERE tirant_eau BETWEEN 10 AND 15;
SELECT * FROM navires WHERE date_arrivee IS NULL;
```

- SELECT choisit les colonnes à retourner ; FROM indique la table source.
- WHERE filtre les **lignes individuelles avant** tout regroupement — ne peut pas utiliser une fonction d'agrégation (COUNT, SUM...) directement.
- IS NULL / IS NOT NULL : seule façon correcte de tester une valeur NULL (= NULL ne fonctionne pas en SQL).

2 Regroupement : GROUP BY, HAVING

```
-- Nombre de navires par port
SELECT port, COUNT(*) AS nb_navires
FROM navires
GROUP BY port;

-- Tirant d'eau moyen par port et par type de navire
SELECT port, type_navire, AVG(tirant_eau) AS tirant_moyen
FROM navires
GROUP BY port, type_navire;

-- HAVING : filtrer APRES le regroupement, sur le resultat de l'agregation
SELECT port, COUNT(*) AS nb_navires
FROM navires
GROUP BY port
HAVING COUNT(*) > 10;
```

- **GROUP BY** regroupe les lignes ayant la même valeur dans la ou les colonnes indiquées, pour appliquer une fonction d'agrégation (COUNT, SUM, AVG, MIN, MAX) à chaque groupe.
- **Différence clé WHERE vs HAVING** : **WHERE** filtre les lignes **avant** le **GROUP BY** (ne connaît pas encore les agrégats); **HAVING** filtre les groupes **après** l'agrégation (peut utiliser COUNT(*), AVG(...), etc. dans sa condition).
- Toute colonne du **SELECT** qui n'est pas agrégée doit apparaître dans le **GROUP BY**.

3 Tri : ORDER BY

```
SELECT nom, tirant_eau FROM navires
ORDER BY tirant_eau DESC; -- décroissant

SELECT nom, tirant_eau FROM navires
ORDER BY port ASC, tirant_eau DESC; -- tri multi-colonnes

SELECT port, COUNT(*) AS nb_navires
FROM navires
GROUP BY port
ORDER BY nb_navires DESC
LIMIT 5; -- top 5 des ports
```

- **ASC** (croissant, par défaut) ou **DESC** (décroissant).
- **ORDER BY** s'exécute en dernier dans la logique de la requête (après **WHERE**, **GROUP BY**, **HAVING**), souvent combiné à **LIMIT** pour un "top N".

4 Jointures : JOIN, INNER JOIN, LEFT JOIN

```
-- INNER JOIN : uniquement les lignes qui correspondent dans les deux tables
SELECT n.nom, n.tirant_eau, p.nom AS port_nom
FROM navires n
INNER JOIN ports p ON n.port_id = p.id;

-- LEFT JOIN : toutes les lignes de la table de gauche (navires),
-- meme sans correspondance dans ports (colonnes de droite a NULL)
SELECT n.nom, p.nom AS port_nom
FROM navires n
LEFT JOIN ports p ON n.port_id = p.id;

-- Jointure sur plusieurs tables
SELECT n.nom, p.nom AS port_nom, e.type_escale
FROM navires n
INNER JOIN ports p ON n.port_id = p.id
INNER JOIN escales e ON e.navire_id = n.id;
```

- **JOIN** seul est équivalent à **INNER JOIN** dans la plupart des SGBD.
- **INNER JOIN** : ne garde que les lignes où la condition **ON** trouve une correspondance dans les deux tables (intersection).
- **LEFT JOIN** (ou **LEFT OUTER JOIN**) : garde **toutes** les lignes de la table de gauche, avec **NULL** pour les colonnes de droite si aucune correspondance n'existe — utile pour repérer les lignes **sans** correspondance (**WHERE p.id IS NULL**).
- Toujours préfixer les colonnes ambiguës par l'alias de table (**n.nom** vs **p.nom**) dès qu'on joint plusieurs tables.

5 Ordre logique d'exécution d'une requête

Ordre	Clause
1	FROM / JOIN
2	WHERE
3	GROUP BY
4	HAVING
5	SELECT
6	ORDER BY
7	LIMIT

- Explique pourquoi `WHERE` ne peut pas utiliser un alias défini dans `SELECT`, et pourquoi `HAVING` peut utiliser un agrégat alors que `WHERE` non.

6 Questions types (révision active)

1. Quelle est la différence fondamentale entre `WHERE` et `HAVING` ?
2. Pourquoi `WHERE tirant_eau = NULL` ne fonctionne-t-il jamais ?
3. Quelle est la différence entre `INNER JOIN` et `LEFT JOIN` ?
4. Comment retrouver les navires qui n'ont **aucune** correspondance dans la table `ports` ?
5. Dans quel ordre logique une requête SQL avec `WHERE`, `GROUP BY`, `HAVING` et `ORDER BY` est-elle exécutée ?
6. Écrivez une requête donnant le top 3 des ports par nombre de navires.

7 Corrigé des questions types

Q1. Différence `WHERE` / `HAVING` ?

`WHERE` filtre les lignes individuelles **avant** tout regroupement — il ne connaît pas encore le résultat d'une agrégation et ne peut donc pas utiliser `COUNT(*)`, `AVG(...)`, etc. `HAVING` filtre les **groupes après** l'exécution du `GROUP BY` et peut donc conditionner sur le résultat d'une fonction d'agrégation.

Q2. Pourquoi `WHERE tirant_eau = NULL` ne fonctionne jamais ?

En SQL, `NULL` représente une valeur inconnue, et toute comparaison avec `NULL` (via `=`, `!=`, etc.) renvoie `NULL` (ni vrai ni faux), jamais `TRUE` — la ligne n'est donc jamais retenue. Il faut utiliser les opérateurs dédiés `IS NULL` ou `IS NOT NULL`.

Q3. Différence `INNER JOIN` / `LEFT JOIN` ?

`INNER JOIN` ne conserve que les lignes qui ont une correspondance dans les **deux** tables (intersection selon la condition `ON`). `LEFT JOIN` conserve **toutes** les lignes de la table de gauche, en complétant par `NULL` les colonnes de la table de droite quand aucune correspondance n'existe.

Q4. Navires sans correspondance dans `ports` ?

```
SELECT n.*
FROM navires n
LEFT JOIN ports p ON n.port_id = p.id
WHERE p.id IS NULL;
```

Le `LEFT JOIN` garde tous les navires, y compris ceux sans port correspondant (colonnes de `ports` à `NULL`) ; le `WHERE p.id IS NULL` isole ensuite uniquement ces lignes sans correspondance.

Q5. Ordre logique d'exécution ?

`FROM/JOIN` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY` → `LIMIT`. Les tables sont d'abord assemblées et filtrées ligne par ligne, puis regroupées et filtrées par groupe, avant que les colonnes finales ne soient calculées et triées.

Q6. Top 3 des ports par nombre de navires :

```
SELECT port, COUNT(*) AS nb_navires
FROM navires
GROUP BY port
ORDER BY nb_navires DESC
LIMIT 3;
```